

Technical Report – Charity Pulse

Project: CharityPulse (PESU_EC_CSE_J_P49)

Team: 8

Team:

Sujoy Sen PES2UG23CS621

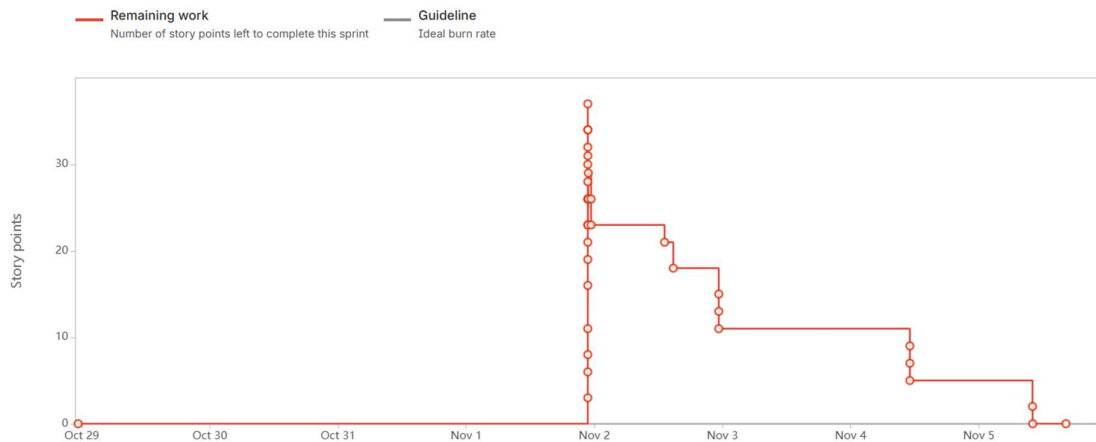
Ishaan Sinha PES2UG23CS921

Ishika Raj PES2UG23CS923

Tanishq Raj PES2UG23CS642

Sprint 1

Burndown Chart:



Test Report

Backend

```
PASS ../../tests/frontend/ProgressBar.test.js (20.767 s)
PASS ../../tests/frontend/login.test.js (20.909 s)
PASS ../../tests/frontend/create-event.test.js (21.011 s)

File      % Stmts   % Branch  % Funcs   % Lines  Uncovered Line #s
-----
All files    94.28     70       100     94.28
 components  100       100       100      100
  ProgressBar.js  100       100       100      100
  pages       93.93     70       100     93.93
  create-event.js 86.66     50       100     86.66  41,48
File      % Stmts   % Branch  % Funcs   % Lines  Uncovered Line #s
-----
All files    94.28     70       100     94.28
 components  100       100       100      100
  ProgressBar.js  100       100       100      100
  pages       93.93     70       100     93.93
  create-event.js 86.66     50       100     86.66  41,48
  components  100       100       100      100
  ProgressBar.js  100       100       100      100
  pages       93.93     70       100     93.93
  create-event.js 86.66     50       100     86.66  41,48
  login.js     100      83.33     100      100      25
-----
  create-event.js 86.66     50       100     86.66  41,48
  login.js     100      83.33     100      100      25
-----
  login.js     100      83.33     100      100      25
-----

Test Suites: 3 passed, 3 total
Tests:       7 passed, 7 total
Snapshots:  0 total
Time:        42.6 s
```

Frontend

```
File      % Stmts   % Branch  % Funcs   % Lines  Uncovered Line #s
-----
All files    94.28     70       100     94.28
 components  100       100       100      100
  ProgressBar.js  100       100       100      100
  pages       93.93     70       100     93.93
File      % Stmts   % Branch  % Funcs   % Lines  Uncovered Line #s
-----
All files    94.28     70       100     94.28
 components  100       100       100      100
  ProgressBar.js  100       100       100      100
  pages       93.93     70       100     93.93
  create-event.js 86.66     50       100     86.66  41,48
  login.js     100      83.33     100      100      25
All files    94.28     70       100     94.28
 components  100       100       100      100
  ProgressBar.js  100       100       100      100
  pages       93.93     70       100     93.93
  create-event.js 86.66     50       100     86.66  41,48
  login.js     100      83.33     100      100      25
  ProgressBar.js  100       100       100      100
  pages       93.93     70       100     93.93
  create-event.js 86.66     50       100     86.66  41,48
  login.js     100      83.33     100      100      25
-----
  create-event.js 86.66     50       100     86.66  41,48
  login.js     100      83.33     100      100      25
-----

Test Suites: 3 passed, 3 total
Tests:       7 passed, 7 total
Snapshots:  0 total
Time:        2.388 s, estimated 22 s
```

Coverage Report:

All files

94.28% Statements 33/35 70% Branches 7/10 100% Functions 8/8 94.28% Lines 33/35

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ▾		Statements ▴ ▾		Branches ▴ ▾		Functions ▴ ▾		Lines ▴ ▾	
pages		93.93%	31/33	70%	7/10	100%	7/7	93.93%	31/33
components		100%	2/2	100%	0/0	100%	1/1	100%	2/2

Retrospection:

Sprint 1: Core Functionality & CI/CD Setup

Goal: Build the Minimum Viable Product (Login, Create Event, Pledge, See Progress) and establish the 5-stage CI/CD pipeline.

What Went Well?

- **Successful MVP:** We successfully built the entire user flow for the core product. An organizer can register, log in, create an event, and a donor can submit a pledge and see the progress bar update in real-time.
- **Git Workflow:** Our branching strategy (feature/ branches, PRs) was a success. We prevented direct commits to main and used Pull Requests for all code reviews.
- **Test-Driven Development (TDD) Success:** Our CI/CD test job successfully caught a major logic bug. The events.test.js file failed because our DELETE /api/events route was not checking for existing pledges (as per SRS CFT-F-002). This proved our testing strategy was working from

Day 1.

- **Good Task Breakdown:** We successfully translated all SRS requirements into actionable user stories in Jira, complete with Epics and Story Points.

What Didn't Go Well? (Problems)

- **Initial Setup Friction (NPM Issues):** We wasted time debugging local server crashes. The backend server failed with Error: Cannot find module 'cors' and later Error: Cannot find module 'winston'. We learned that because we have two package.json files (frontend/backend), we must run npm install in both src/backend and src/frontend directories.
- **CI/CD Test Failures (Import Stack):** Our first integration tests failed the whole pipeline. auth.test.js (which was correct) failed because it imported server.js, which imported events.js, which imported a non-existent logger.js. This was a key learning experience.
- **Workload Imbalance:** As the sprint progressed, it became clear the workload was not evenly distributed. The backend role (Ishaan) was a bottleneck. Almost every frontend story required a new API endpoint, giving Ishaan twice as many PRs and tasks as the frontend developers.

What Did We Learn?

- A "full-stack" story (like "Create Event") is really two separate tasks (frontend UI + backend API) and must be planned as such.
- A failing test in the pipeline is a success, not a problem. It's the system working as designed.
- The CI/CD test environment is "dumber" than our local one. It crashes on simple import errors that our local machines might ignore, forcing us to write cleaner code.

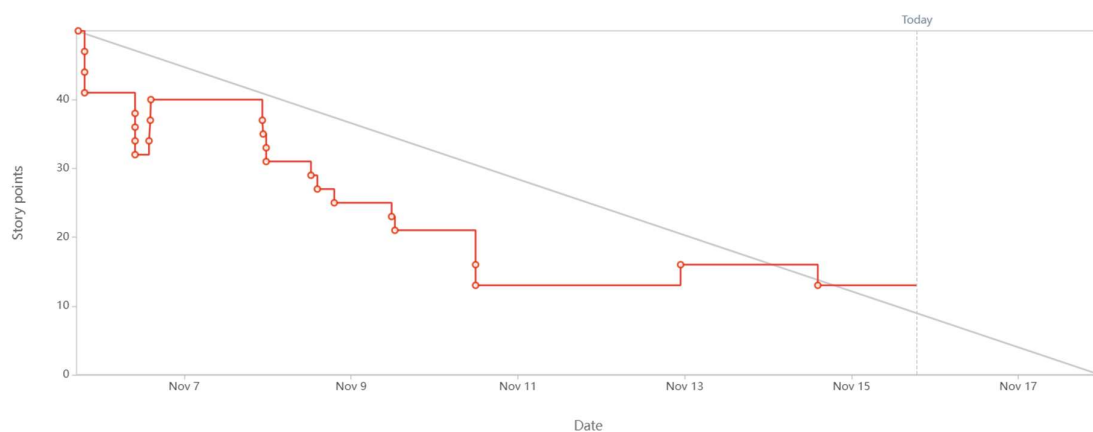
Action Items (for Sprint 2)

- **Redistribute Workload:** Re-plan Sprint 2 to account for the backend bottleneck. Frontend developers (Sujoy, Ishika) will be the "Lead" on a feature, and Ishaan will be assigned the specific backend sub-tasks.

- Backend-Heavy Stories: Ishaan will focus on complex, backend-only stories (MFA, Account Lockout), while Tanishq will take the lead on all CI/CD and Quality Gate stories (CEFTS-36, CEFTS-37).
- Fix CI Crashes Immediately: If the test suite fails to run (like the logger.js crash), that PR becomes the number one priority to fix and merge, as it blocks all other tests.

Sprint 2

1. Burndown Chart



2. Test Report

Frontend

```
PASS ../../tests/frontend/ProgressBar.test.js (26.886 s)
PASS ../../tests/frontend/CloseEventButton.test.js (26.971 s)
PASS ../../tests/frontend/AdminPanel.test.js (26.994 s)
PASS ../../tests/frontend/setup-mfa.test.js (27.034 s)
PASS ../../tests/frontend/summary-page.test.js (27.129 s)
PASS ../../tests/frontend/forgot-password.test.js (27.135 s)
PASS ../../tests/frontend/DonationChart.test.js (27.167 s)
PASS ../../tests/frontend/EventCard.test.js (27.21 s)
PASS ../../tests/frontend/register.test.js (27.223 s)
PASS ../../tests/frontend/login.test.js (27.271 s)
```

```
Test Suites: 10 passed, 10 total
Tests:       24 passed, 24 total
Snapshots:   0 total
Time:        64.653 s
Ran all test suites.
```

Backend

```
Test Suites: 8 passed, 8 total
Tests:       40 passed, 40 total
Snapshots:   0 total
Time:        9.721 s
Ran all test suites.
```

3. Coverage Report

Frontend

PASS

../../../../tests/frontend/DonationChart.test.js

PASS

../../../../tests/frontend/ProgressBar.test.js

PASS

../../../../tests/frontend/CloseEventButton.test.js

PASS

../../../../tests/frontend/AdminPanel.test.js

PASS

../../../../tests/frontend/summary-page.test.js

PASS

../../../../tests/frontend/setup-mfa.test.js

PASS

../../../../tests/frontend/EventCard.test.js

PASS

../../../../tests/frontend/forgot-password.test.js

PASS

../../../../tests/frontend/register.test.js

PASS

../../../../tests/frontend/login.test.js

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	87.11	67.6	93.75	90.72	
components	100	90	100	100	
CloseEventButton.js	100	83.33	100	100	11
DonationChart.js	100	100	100	100	
EventCard.js	100	100	100	100	
ProgressBar.js	100	100	100	100	
pages	92	65	100	94.73	
forgot-password.js	87.5	50	100	93.33	26
login.js	100	90	100	100	38
register.js	100	87.5	100	100	36
setup-mfa.js	80.64	42.85	100	86.2	16-17,28,53
pages/admin	66.66	61.53	66.66	68	
users.js	66.66	61.53	66.66	68	28-40,72
pages/reports/summary	80.95	62.5	100	94.11	
[id].js	80.95	62.5	100	94.11	20

Test Suites: 10 passed, 10 total

Tests: 24 passed, 24 total

Snapshots: 0 total

Time: 7.015 s, estimated 28 s

Ran all test suites.

All files

87.11% Statements 342/393 67.6% Branches 46/71 93.75% Functions 39/42 90.72% Lines 337/354

Press n or j to go to the next uncovered block, b, p or k for the previous block

Filter

File	Statements	Branches	Functions	Lines
components	100%	15/15	90%	14/14
pages	92%	92/100	65%	90/95
pages/admin	66.66%	18/27	61.53%	17/25
pages/reports/summary	80.95%	17/21	62.5%	16/17

All files									
87.11% Statements 342/393 67.6% Branches 46/71 93.75% Functions 39/42 90.72% Lines 337/354									
Press n or j to go to the next uncovered block, b, p or k for the previous block									
Filter									
File	Statements	Branches	Functions	Lines					
components		100%	15/15	90%	9/10	100%	5/5	100%	14/14
pages		92%	92/100	65%	26/40	100%	18/18	94.73%	90/95
pages/admin		66.66%	18/27	61.53%	8/13	66.66%	4/6	68%	17/25
pages/reports/summary		80.95%	17/21	62.5%	5/8	100%	3/3	94.11%	16/17

Backend

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	76.13	64.77	84.84	76.72	
backend	100	50	100	100	
logger.js	100	50	100	100	19
server.js	100	100	100	100	
backend/middleware	78.26	66.66	100	78.26	
admin.js	81.81	100	100	81.81	16-17
auth.js	75	50	100	75	10,19-20
backend/models	100	100	100	100	
Event.js	100	100	100	100	
Pledge.js	100	100	100	100	
User.js	100	100	100	100	
backend/routes	74.06	65	83.87	74.7	
admin.js	79.16	50	100	79.16	16-17,26,35-36
auth.js	58.47	44.44	62.5	60	62-63,83-150,156-197,234-235,267-268
events.js	81.52	93.75	80	82.22	48-51,84-85,94-101,129-130,160-161,186-187
mfa.js	78.37	50	100	78.37	27,33-34,45,64-69
pledges.js	91.66	100	100	91.66	50-51
reports.js	82.69	62.5	100	82	15,52-53,62,79-80,90,103-104
Test Suites: 8 passed, 8 total					
Tests: 40 passed, 40 total					
Snapshots: 0 total					
Time: 9.721 s					
Ran all test suites.					

All files
76.13% Statements 342/395 66.66% Branches 64/96 84.84% Functions 29/33 76.72% Lines 308/399

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
backend	100%	19/19	50%	19/19
backend/middleware	78.26%	18/23	83.33%	18/23
backend/models	100%	9/9	100%	9/9
backend/routes	74.06%	257/347	65.85%	254/340

Retrospection:

Goal: Complete all remaining features (Reports, Admin, Security) and meet the final quality gates (75% Coverage, 7.5 Lint Score, E2E Tests, Deployment Artifact).

What Went Well?

- Improved Workload Distribution: The Action Item from Sprint 1 was a complete success. The new plan (assigning "leads" and "sub-tasks") worked perfectly. We had no bottlenecks, and work progressed in parallel.
- CI/CD Pipeline as a Tool: Our pipeline was our most valuable team member. It caught numerous bugs, including: ReferenceError: Pledge is not defined in reports.test.js.
- A logic bug in the reports.js aggregation (Expected: "Charlie", Received: undefined).
- A test bug in the CSV export (missing quotes).

Full 6-Stage Pipeline: We successfully implemented all required CI/CD stages, including the new e2e-test (Cypress), security (npm audit), and deploy-artifact jobs.

Met All Quality Gates: By tracking the coverage and lint jobs, we successfully met our Sprint 2 goals for 75% test coverage and a high lint score.

Cross-Team Collaboration: When `admin.test.js` failed, the team correctly identified that the `admin.js` route file hadn't even been created yet, and the `User.js` model was missing the `isRevoked` field. This showed excellent full-stack debugging.

What Didn't Go Well? (Problems)

- Major CI/CD Pipeline Failure (Artifacts): Our entire pipeline suddenly broke. All jobs started failing with Error: ... actions/upload-artifact: v3 is deprecated. We learned that our CI/CD dependencies can become outdated. We had to fix this by updating all uses: lines from `@v3` to `@v4`. This was a critical, unexpected bug.
- Local Environment Errors: We lost time to a local-only bug. `npm run lint` failed on a local machine (Error: Failed to load plugin '@next/next') but passed in the pipeline. This forced us
- to learn how to fix corrupted dependencies by deleting `node_modules` and `package-lock.json` and running `npm install` again.
- Minor Linting Bugs: We had a pipeline failure on a simple linting error (react/no-escaped-entities) because of an apostrophe in "We'll". This was a quick fix (') but a reminder to run lint locally before pushing.

What Did We Learn?

- The CI/CD pipeline itself is code and must be maintained. It is not "set it and forget it."
- "It works on my machine" is not an excuse. The CI/CD pipeline is the single source of truth.
- A "bug-fix loop" (push -> fail -> fix -> push) is a normal and healthy part of the CI/CD process.
- The team is now fully proficient in reading CI/CD error logs to find the exact file, line, and error message of a bug.

